

# Geog4/6300: Lab 1

---

## Loading data into R, data transformation, and summary statistics

---

Your name: William Burgett-Merkel

### Overview and lab criteria:

This lab is intended to assess your ability to use R to load data and to generate basic descriptive statistics. For this lab to be marked complete, the following criteria must be met:

4. Identify and apply appropriate data filtering and cleaning strategies to prepare datasets for analysis. (Task 2)
5. Effectively interpret the code you create, explaining in plain language what each step does to the data. (Task 6)
6. Identify and use appropriate external documentation — including package references, help files, and peer resources — to learn and apply unfamiliar functions or methods. (Task 7)
7. Filter, aggregate, and transform datasets using grouping and summary operations to answer specific analytical questions. (Tasks 1 & 5)
8. Reshape data between wide and long formats to meet the requirements of different analytical or visualization tasks. (Task 4)
9. Create effective data visualizations across multiple chart types (line, scatter, histogram, Q-Q plot), applying appropriate aesthetic choices such as color, grouping, and labeling. (Task 3 & 5)

### Data:

You'll be using monthly weather data from the Daymet climate database (<http://daymet.ornl.gov>) for all counties in the United States over an 12-year period (2010-2021). These data are available on the GitHub repo for our course. The following variables are provided:

- `cty_txt` : Code for joining to census data
- `year` : Year of observation (with an initial "Y" to make it a character)
- `month` : Month of observation (1 = Jan, 2 = Feb, etc.)

- `median_tmax` : Median maximum recorded temperature (Celsius)
- `median_tmin` : Median minimum recorded temperature (Celsius)
- `sum_prctp` : Total recorded precipitation for the month (mm)
- `cty_name` : Name of the county
- `state` : State of the county
- `region` : Census region (map: [https://www2.census.gov/geo/pdfs/maps-data/maps/reference/us\\_regdiv.pdf](https://www2.census.gov/geo/pdfs/maps-data/maps/reference/us_regdiv.pdf))
- `division` : Census division
- `x` : Longitude of the county centroid
- `y` : Latitude of the county centroid

These labs are meant to be done collaboratively, but your final submission should demonstrate your own original thought (don't just copy your classmate's work or turn in identical assignments). Your answers to the lab questions should be typed in this Quarto template. You'll then render the document to a GitHub markdown document and upload it to your class GitHub repo.

### Procedure:

Load the tidyverse package and import the data:

```
library(tidyverse)

daymet_data <- read_csv("data/daymet_monthly_median_2010-2021.csv")
```

We can look at the first few rows of the dataset using the `head()` function. We also use `kable` to format the output as a readable table..

```
kable(head(daymet_data))
```

| cty_txt | year  | month | median_tmax | median_tmin | sum_prctp | cty_name    |
|---------|-------|-------|-------------|-------------|-----------|-------------|
| G02060  | Y2010 | 1     | -4.27       | -10.83      | 10.04     | Bristol Bay |
| G02185  | Y2010 | 1     | -20.73      | -28.20      | 0.00      | North Slope |
| G02180  | Y2010 | 1     | -16.50      | -23.72      | 5.75      | Nome        |

| cty_txt | year  | month | median_tmax | median_tmin | sum_prcp | cty_name          |
|---------|-------|-------|-------------|-------------|----------|-------------------|
| G02050  | Y2010 | 1     | -11.20      | -18.90      | 24.55    | Bethel            |
| G02261  | Y2010 | 1     | -13.93      | -20.03      | 15.84    | Valdez-Cordova    |
| G02170  | Y2010 | 1     | -5.10       | -12.42      | 35.84    | Matanuska-Susitna |

There are a lot of observations here, 452,448 to be exact. To get a better grasp on the data, we can use `group_by()` and `summarise()` from the tidyverse package. This will allow us to identify the mean value for each year by county across the study period.

## Task 1

Use `group_by()` and `summarise()` to calculate the mean minimum temperature for each year by county across all months, also including State and Region as grouping variables. Your resulting dataset should show the value of `tmin` for each county in each year. Use the `kable()` and `head()` functions as shown above to call the resulting table.

```
daymet_data_mutated<-daymet_data |>
  mutate(year = str_remove_all(year, "Y"))

daymet_data_Task1<-daymet_data_mutated |>
  group_by(year, cty_name, state, region) |>
  summarize(mean_tmin=mean(median_tmin))
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by year, cty_name, state, and region.
i Output is grouped by year, cty_name, and state.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(year, cty_name, state, region))` for per-operation
  grouping (`?dplyr::dplyr_by`) instead.
```

```
kable(head(daymet_data_Task1))
```

| year | cty_name  | state          | region         | mean_tmin |
|------|-----------|----------------|----------------|-----------|
| 2010 | Abbeville | South Carolina | South Region   | 10.009167 |
| 2010 | Acadia    | Louisiana      | South Region   | 14.135417 |
| 2010 | Accomack  | Virginia       | South Region   | 10.250417 |
| 2010 | Ada       | Idaho          | West Region    | 4.771667  |
| 2010 | Adair     | Iowa           | Midwest Region | 3.143333  |
| 2010 | Adair     | Kentucky       | South Region   | 6.894167  |

## Task 2

---

*Let's shift to the state level, focusing on those in the South Region. Filter the original data frame ( `daymet_data` ) to just include counties in this region. Then calculate the mean minimum temperature by year for each state. For an optional extra challenge, use the `round()` function to include only 1 decimal point. Use `kable()` and `head()` to call the first few lines of the resulting table.*

```
daymet_data_mutated<-daymet_data |>
  mutate(year = str_remove_all(year, "Y"))

daymet_data_South_Region<-daymet_data_mutated |>
  filter(region=="South Region")

daymet_data_Task2<-daymet_data_South_Region |>
  group_by(year, state) |>
  summarize(mean_tmin=mean(median_tmin))
```

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by year and state.
i Output is grouped by year.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(year, state))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
daymet_data_Task2$mean_tmin<-round(daymet_data_Task2$mean_tmin, digits = 1)

kable(head(daymet_data_Task2))
```

| year | state                | mean_tmin |
|------|----------------------|-----------|
| 2010 | Alabama              | 10.2      |
| 2010 | Arkansas             | 10.1      |
| 2010 | Delaware             | 8.7       |
| 2010 | District of Columbia | 9.2       |
| 2010 | Florida              | 14.2      |
| 2010 | Georgia              | 10.3      |

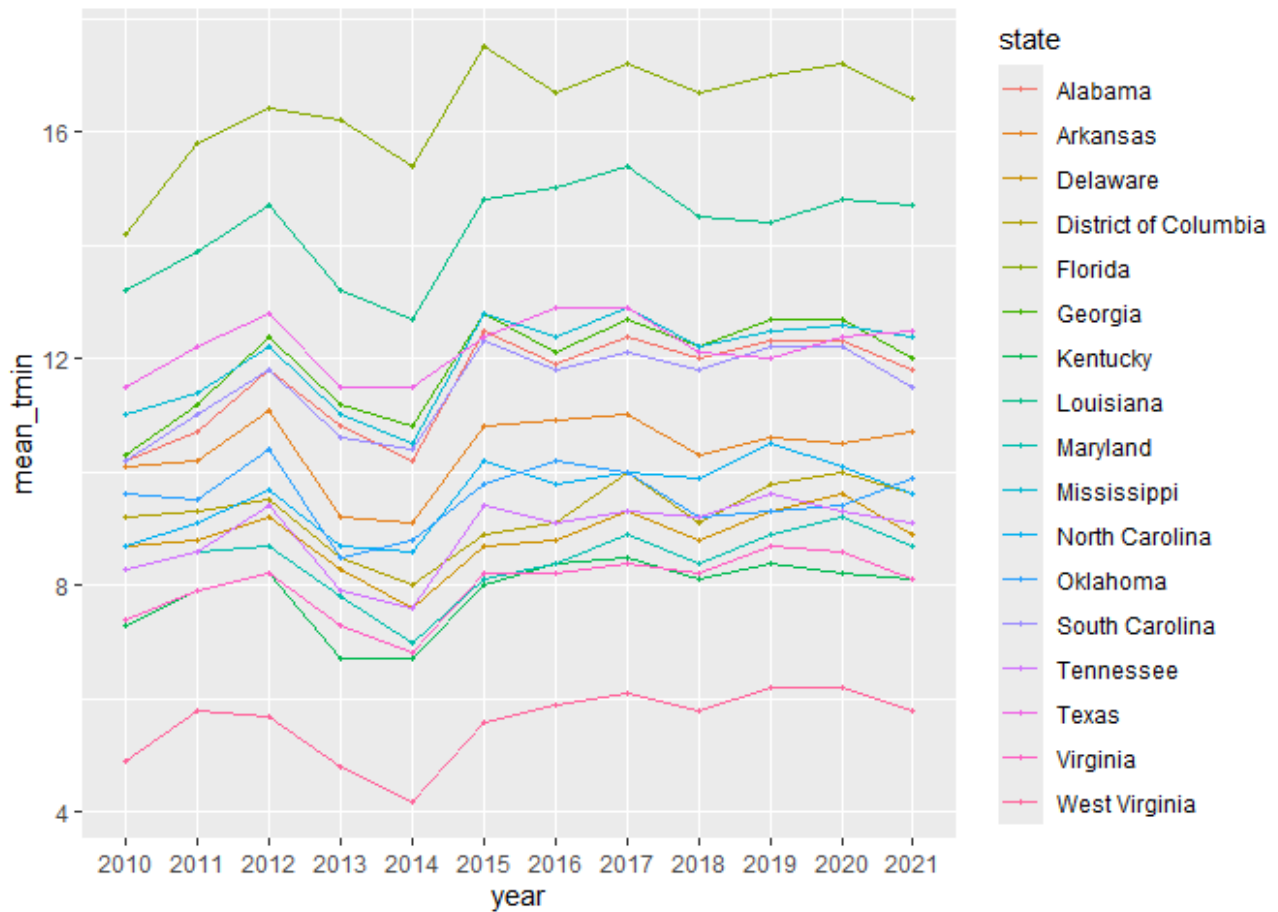
## Task 3

---

*To visualize the trends, we could use ggplot to visualize change in mean temperature over time. Create a line plot ( `geom_Line` ) showing the state means you calculated in task 2. Use the `color` parameter to show separate colors for each state. You may also need to define the state as a group in the aesthetic parameter.*

```
ggplot(daymet_data_Task2,
       aes(x=year,
           y=mean_tmin,
           group=state,
           color=state))+
  geom_line(size=.5)+
  geom_point(size=.75)
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.  
 i Please use `linewidth` instead.



## Task 4

If you wanted to look at these data as a table, you'd need to have it in wide format. Use the `pivot_wider()` function to create a wide-format version of the data frame you created in task 2. In this case, the rows should be states, the columns should be the years, and the data in those columns should be mean minimum temperatures. Then call the whole table using `kable()`.

```
daymet_data_Task4<-daymet_data_Task2 |>
  pivot_wider(values_from = mean_tmin,
              names_from=year)

kable(daymet_data_Task4)
```

| state    | 2010 | 2011 | 2012 | 2013 | 2014 | 2015 | 2016 | 2017 | 2018 |
|----------|------|------|------|------|------|------|------|------|------|
| Alabama  | 10.2 | 10.7 | 11.8 | 10.8 | 10.2 | 12.5 | 11.9 | 12.4 | 12.0 |
| Arkansas | 10.1 | 10.2 | 11.1 | 9.2  | 9.1  | 10.8 | 10.9 | 11.0 | 10.3 |
| Delaware | 8.7  | 8.8  | 9.2  | 8.3  | 7.6  | 8.7  | 8.8  | 9.3  | 8.8  |

| state                | 2010 | 2011 | 2012 | 2013 | 2014 | 2015 | 2016 | 2017 | 2018 |
|----------------------|------|------|------|------|------|------|------|------|------|
| District of Columbia | 9.2  | 9.3  | 9.5  | 8.5  | 8.0  | 8.9  | 9.1  | 10.0 | 9.1  |
| Florida              | 14.2 | 15.8 | 16.4 | 16.2 | 15.4 | 17.5 | 16.7 | 17.2 | 16.7 |
| Georgia              | 10.3 | 11.2 | 12.4 | 11.2 | 10.8 | 12.8 | 12.1 | 12.7 | 12.2 |
| Kentucky             | 7.3  | 7.9  | 8.2  | 6.7  | 6.7  | 8.0  | 8.4  | 8.5  | 8.1  |
| Louisiana            | 13.2 | 13.9 | 14.7 | 13.2 | 12.7 | 14.8 | 15.0 | 15.4 | 14.5 |
| Maryland             | 8.3  | 8.6  | 8.7  | 7.8  | 7.0  | 8.1  | 8.4  | 8.9  | 8.4  |
| Mississippi          | 11.0 | 11.4 | 12.2 | 11.0 | 10.5 | 12.8 | 12.4 | 12.9 | 12.2 |
| North Carolina       | 8.7  | 9.1  | 9.7  | 8.7  | 8.6  | 10.2 | 9.8  | 10.0 | 9.9  |
| Oklahoma             | 9.6  | 9.5  | 10.4 | 8.5  | 8.8  | 9.8  | 10.2 | 10.0 | 9.2  |
| South Carolina       | 10.2 | 11.0 | 11.8 | 10.6 | 10.4 | 12.3 | 11.8 | 12.1 | 11.8 |
| Tennessee            | 8.3  | 8.6  | 9.4  | 7.9  | 7.6  | 9.4  | 9.1  | 9.3  | 9.2  |
| Texas                | 11.5 | 12.2 | 12.8 | 11.5 | 11.5 | 12.4 | 12.9 | 12.9 | 12.1 |
| Virginia             | 7.4  | 7.9  | 8.2  | 7.3  | 6.8  | 8.2  | 8.2  | 8.4  | 8.2  |
| West ... ..          | 4.9  | 5.8  | 5.7  | 4.8  | 4.2  | 5.6  | 5.9  | 6.1  | 5.8  |

## Task 5

Let's assess the relationship of heat and precipitation by region. Returning to the original dataset, create a data frame that shows the mean maximum temperature and mean precipitation for all states in 2015, also including region as a subgroup in your `group_by`. Then use `ggplot` to create a scatterplot ( `geom_point` ) for these two variables, coloring the points using the region variable.

```
daymet_data_mutated<-daymet_data |>
  mutate(year = str_remove_all(year, "Y"))

daymet_data_Task5<-daymet_data_mutated |>
  filter(year=="2015") |>
```

```
group_by(year, state, region) |>
summarize(mean_tmax=mean(median_tmax),mean_prctp=mean(sum_prctp))
```

`summarise()` has regrouped the output.

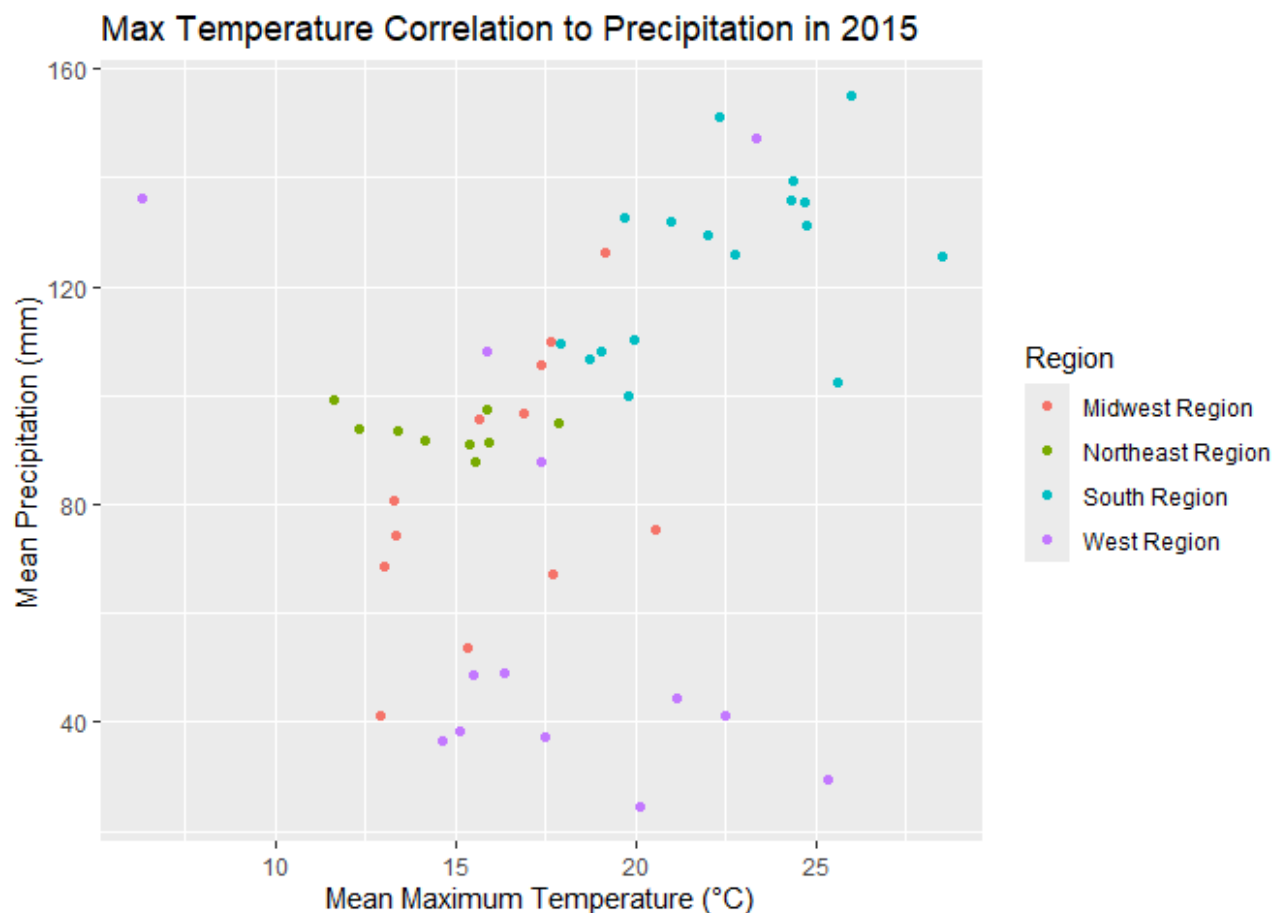
• Summaries were computed grouped by year, state, and region.

• Output is grouped by year and state.

• Use `summarise(.groups = "drop\_last")` to silence this message.

• Use `summarise(.by = c(year, state, region))` for per-operation grouping  
(`?dplyr::dplyr\_by`) instead.

```
ggplot(daymet_data_Task5,
       aes(x=mean_tmax,
           y=mean_prctp,
           color=region))+
labs(title="Max Temperature Correlation to Precipitation in 2015",
     x="Mean Maximum Temperature (°C)",y="Mean Precipitation (mm)",
     color="Region")+
geom_point()
```



## Task 6

---

*In the space below, explain what each function in your code for task 5 does to the dataset in plain English.*

My first line of code takes the year data, which is formatted for example as Y2015, and makes it just 2015. I did this to ensure I didn't have any errors converting it or doing analysis. To do this, I took the initial daymet\_data file and created a mutated file, where I used the mutate() function to remove "Y" from the year column in the daymet\_data.

Next I had to create a new Data Frame. To do this, the first function I employed was the filter() function on a new dataset specifically for Task 5 (daymet\_data\_Task5). This function filtered the year variable for just 2015. Then I used the group\_by() function to get all the additional values required by the task, which were year, state, and region. I kept year even in the group\_by() function as it allows the dataset to be used in the future, especially if it were to be joined datasets for other years. Lastly, I used the summarize() function to calculate the mean max temperature and mean precipitation for every state in 2015.

Lastly, to plot the scatterplot I called upon ggplot(). First, I had to call my new data frame (daymet\_data\_Task5). Then I set the x and y variables to mean max temperature and mean precipitation respectively and set the points to be differentiated by color based on what region the state is in. Again, for good practice, I added an extra step in using labs() to rename the title, x and y axis, and legend. To finish the plot, I simply used geom\_point() to actually generate a scatterplot.

## Task 7

---

*The dplyr package also includes across function. Use ?across on the R command line to open the documentation for this function. In the space below, explain what it does in your own words. Then interpret the way the across function is used below, going line by line within the function.*

```
state_2015 <- daymet_data %>%  
  filter(year == "Y2015") %>%  
  group_by(region, state) %>%  
  summarise(  
    across(  
      c(median_tmax, sum_prcp),  
      mean,  
      na.rm = TRUE,  
      .names = "mean_{.col}"
```

```
)  
)
```

Warning: There was 1 warning in `summarise()`.

• In argument: `across(c(median\_tmax, sum\_prcp), mean, na.rm = TRUE, .names = "mean\_{.col}")`.

• In group 1: `region = "Midwest Region"`, `state = "Illinois"`.

Caused by warning:

! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.

Supply arguments directly to `.fns` through an anonymous function instead.

```
# Previously
```

```
across(a:b, mean, na.rm = TRUE)
```

```
# Now
```

```
across(a:b, \(x) mean(x, na.rm = TRUE))
```

`summarise()` has regrouped the output.

• Summaries were computed grouped by region and state.

• Output is grouped by region.

• Use `summarise(.groups = "drop\_last")` to silence this message.

• Use `summarise(.by = c(region, state))` for per-operation grouping (`?dplyr::dplyr_by``) instead.

The `across()` function allows people to run the same function for different columns of data within the same dataset in the same function. In the code for task 7, first the dataset was filtered for only the year 2015, then the data was grouped to only include the region and state data for each row. The `across()` function was used within the `summarize()` function to find the mean for both the median maximum temperature and sum of precipitation column within one line of code. This is a much more efficient way of getting the mean than writing two mean functions separated by a comma in the same `summarize()` function as I did in task 5. Next the `"na.rm = True"` function gets rid of any NA data, and the `".names = "mean_{.col}"` function creates a name for the new column within the new data frame.

## Challenge Question

---

In class, we covered ways of working with the Daymet API. Create a script below that uses the **daymetr** package to download data from Daymet for a place (or places) of your choosing. Then visualize the temporal pattern for a variable of your choosing in this place, similar to what you did in question 4. Use a dplyr function (`mutate()`, `summarise()`, `filter()`, etc.) to do any needed data wrangling and create a visual using ggplot.

In addition to this code, write a short summary of a pattern that's evident in the data you visualized.

```
# Your code goes here
```

*Explanation goes here.*

## Final Submission Stuff

---

### Disclosure of Assistance

Besides class materials, what other sources of assistance did you use while completing this lab? These can include input from classmates, relevant material identified through web searches (e.g., Stack Overflow), or assistance from ChatGPT or other AI tools. How did these sources support your own learning in completing this lab?

I used Google AI to help me when I ran into errors that I did not know how to fix. An example was in task 5, as I was getting an error that stated the "sum\_prcp" variable did not exist even though it obviously does. AI helped me understand how R runs the chunk and why it could not find it (I was running summarize() twice, which created a new data frame, and when I ran the second summarize() the sum\_prcp was not in it). I also used it to explain the difference in "values\_from" and "names\_from" for task 4, as states kept ending up as columns. Lastly, I used it to understand how the round() function works, because when I was running it like how it is in the help section (round(x, digits = 0, ...)), it was not working. AI explained to me how they worked and told me I needed to overwrite the column, which I figured out how to do on my own.

Using AI helps me get a better understanding of the code, specifically when errors don't make sense. I can get it to break down how R processes code so I can better write my code. It also allows me to learn the processes behind code and helps me understand my errors so I can fix them without spending a lot of time trying to understand the error. It just can be a general time saver compared to stack overflow or other coding help websites.

### Lab Reflection

How do you feel about the work you did on this lab? Was it easy, moderate, or hard? What are the biggest things you learned by completing it?

I honestly thought the lab was not bad, when I did my initial pass through I thought it would be more difficult but once I sat down and thought about how I should go about each question I got through it pretty easily. The biggest thing I learned is the formatting for

ggplot(), and just how streamlined the process can be for making charts in R.